HYPERION

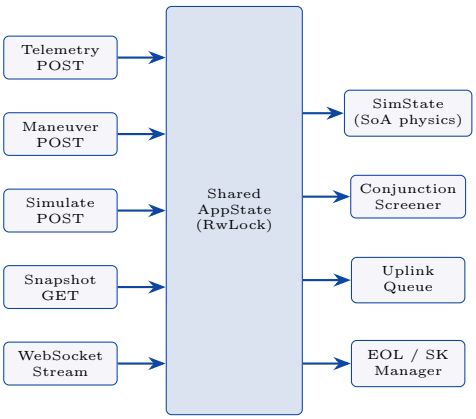
Autonomous Constellation Manager

National Space Hackathon 2026 · Technical Report

Stack: Rust 1.94.0 (Axum, Rayon, Tokio) · **Frontend:** TypeScript + Bun · **Deployment:** Docker on `ubuntu:22.04`

System Architecture

Hyperion is a fully autonomous, ground-based constellation management system built around a **lock-free, async shared state** architecture. All API handlers share a single `Arc<RwLock<AppState>>` that wraps the physics engine, conjunction cache, uplink queue, and telemetry clock—eliminating inter-process serialization overhead while preserving Rust's memory-safety guarantees at zero runtime cost.



Structure of State-of-Array (SoA) Physics Engine

The core `SimState` stores all object fields in parallel `Vec<f64>` arrays (position, velocity, mass, nominal slot, etc.) rather than a `Vec<Satellite>` AoS layout. This maximizes **SIMD friendliness** and **cache locality** during the Rayon-parallelised RK4 sweep—the hottest path in the system.

Orbital Propagation: RK4 + J2

Equations of Motion

Each object is governed by the second-order ODE

$$\ddot{\mathbf{r}} = -\frac{\mu}{|\mathbf{r}|^3}\,\mathbf{r} + \mathbf{a}_{J_2} \tag{1}$$

where $\mu = 398600.4418\ \mathrm{km}^3\mathrm{s}^{-2}$ and the oblateness perturbation is

$$\mathbf{a}_{J_2} = \frac{3}{2}\frac{J_2 \mu R_E^2}{|\mathbf{r}|^5}\begin{pmatrix} x\left(5\frac{z^2}{|\mathbf{r}|^2} - 1\right) \\ y\left(5\frac{z^2}{|\mathbf{r}|^2} - 1\right) \\ z\left(5\frac{z^2}{|\mathbf{r}|^2} - 3\right) \end{pmatrix} \tag{2}$$

with $R_E = 6378.137$ km, $J_2 = 1.08263 \times 10^{-3}$.

Fourth-Order Runge–Kutta Integration

The state vector $\mathbf{s} = [\mathbf{r}, \mathbf{v}]^\top$ is advanced by a classical RK4 step with **30-second sub-steps**:

$$\mathbf{s}_{n+1} = \mathbf{s}_n + \tfrac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \tag{3}$$

Sub-stepping bounds the local truncation error to $\mathcal{O}(h^5) \approx 1$ m even on 1-hour simulation ticks.

Parallel Propagation via Rayon

All $N_{\mathrm{obj}} \approx 10\,050$ objects are propagated in a single `par_iter()` sweep using Rayon's work-stealing thread pool, achieving near-linear scaling on multi-core hardware.

Nominal Slot Tracking & Station-Keeping

To measure operational drift for telemetry reporting, each satellite's *nominal slot* (n_x, n_y, n_z) is propagated using **pure two-body** mechanics. The real trajectory accumulates perturbations and maneuver offsets, allowing the system to quantify displacement for the frontend visualizer.

However, to minimize computational overhead during autonomous operations, actual station-keeping is achieved proactively via **Inverted Phasing Recovery**. Rather than continuously calculating bounding-box intersections against the nominal slot, the engine autonomously schedules an inverse Δv burn post-evasion to nullify the velocity debt and natively restore the original orbital period.

Conjunction Assessment

Algorithm Selection

> The naïve $O(N^2)$ grid approach required **32 seconds** per screening cycle on a 10 000-object field. Hyperion's revised pipeline completes in < 100 **ms**.

For $N_{\mathrm{sat}} \leq 500$ (the nominal scenario), Hyperion replaces the grid with a **direct-pair enumeration** of only $N_{\mathrm{sat}} \times N_{\mathrm{deb}} = 50 \times 10{,}000 = 500{,}000$ pairs, eliminating all HashMap insertions from the hot path. Above 500 satellites the system automatically falls back to the spatial grid.

Five-Phase Filter Pipeline

P1. Altitude Band — Compute perigee/apogee once per object via the vis-viva equation. $O(N_{\text{sat}} + N_{\text{deb}})$.

P2. Apogee/Perigee Overlap — Reject pairs whose altitude bands cannot intersect: $p_{\text{sat}} > a_{\text{deb}} + \epsilon$ or vice-versa.

P3. Altitude Midpoint Gate — Discard pairs whose band midpoints differ by more than the combined half-widths plus $d_{\text{slack}} = 500$ km.

P4. Linear CPA Filter — Compute the closed-form linear closest-point-of-approach time $t^* = -\frac{\Delta\mathbf{r}\cdot\Delta\mathbf{v}}{|\Delta\mathbf{v}|^2}$ and reject if the resulting distance exceeds d_{slack}.

P5. Brent TCA/PCA — Run the high-fidelity RK4 + Brent minimisation only on the surviving pairs ($\ll 1\%$ of all pairs).

Multi-Window Brent Minimisation

The distance function $d(t) = |\mathbf{r}_{\text{sat}}(t) - \mathbf{r}_{\text{deb}}(t)|$ is minimised over the 24-hour horizon using **Brent's method** (1973)—combining golden-section reliability with parabolic interpolation speed. The horizon is partitioned into windows of half the mean orbital period to avoid missing secondary close approaches:

$$T_{\text{window}} = \min\left(\frac{T_{\text{period}}}{2},\ T_{\text{horizon}}\right) \tag{4}$$

Brent converges in < 30 iterations to a tolerance of 10^{-4} s. Each surviving pair is evaluated independently, enabling full **thread-level parallelism** via `thread::scope`.

Autonomous Collision Avoidance

Threat Detection & Throttling

The autonomous evasion loop runs on every telemetry ingest and simulation step, but is **throttled to 15 simulation-seconds** between full screenings to prevent CPU starvation during high-frequency telemetry bursts. Any conjunction with $d_{\text{PCA}} < 100$ m triggers an emergency burn sequence.

Prograde/Retrograde Trial Ephemeris

For each critical threat the system evaluates two candidate burns in the satellite's local **RTN frame**:

$$\Delta\mathbf{v}_{\text{pro}} = [0,\ +\Delta v_{\max},\ 0]_{\text{RTN}}, \quad \Delta\mathbf{v}_{\text{ret}} = [0,\ -\Delta v_{\max},\ 0]_{\text{RTN}} \tag{5}$$

Both are converted to ECI via the rotation matrix built from the instantaneous $\hat{R}$, $\hat{T}$, $\hat{N}$ unit vectors. Each candidate is propagated through a *virtual clone* and its post-burn PCA is computed with Brent TCA. The candidate achieving the greater miss distance is selected.

Preference for in-plane (T-direction) burns is deliberate: *out-of-plane* maneuvers are 5–10× more fuel-expensive for equivalent collision avoidance benefit.

RTN Frame Construction

$$\hat{R} = \frac{\mathbf{r}}{|\mathbf{r}|}, \quad \hat{N} = \frac{\mathbf{r}\times\mathbf{v}}{|\mathbf{r}\times\mathbf{v}|}, \quad \hat{T} = \hat{N}\times\hat{R} \tag{6}$$

The ECI $\Delta\mathbf{v}$ submitted to the maneuver queue is

$$\Delta\mathbf{v}_{\text{ECI}} = \Delta v_R\,\hat{R} + \Delta v_T\,\hat{T} + \Delta v_N\,\hat{N} \tag{7}$$

Station-Keeping Recovery

Immediately after scheduling the evasion burn, Hyperion schedules a **recovery burn** with the exact inverse $\Delta\mathbf{v}$ vector, timed at

$$t_{\text{recovery}} = t_{\text{now}} + t_{\text{TCA}} + 3600\ \text{s} \tag{8}$$

Caveat: Secondary Conjunctions & Threat Masking

While the immediate scheduling of an inverted recovery burn guarantees long-term station-keeping, it introduces a potential vulnerability regarding **Secondary Conjunctions**.

Propulsion & Fuel Accounting

Tsiolkovsky Rocket Equation

Fuel consumed per burn is computed exactly:

$$\Delta m = m_{\text{current}}\left(1 - e^{-\frac{|\Delta v|}{I_{\text{sp}}\,g_0}}\right) \tag{9}$$

with $I_{\text{sp}} = 300$ s, $g_0 = 9.80665\ \text{m}\,\text{s}^{-2}$. Because mass decreases after each burn, subsequent maneuvers consume proportionally less propellant—Hyperion re-evaluates m_{current} at every burn in a sequence, not once up-front.

End-of-Life Management

When residual fuel falls below 2.5 kg (5% of initial propellant), the satellite is flagged `is_eol = true` and a **retrograde deorbit burn** is autonomously scheduled:

$$\Delta v_{\text{EOL}} = \min\left(I_{\text{sp}}\,g_0\,\ln\frac{m_{\text{wet}}}{m_{\text{dry}}},\ \Delta v_{\max}\right) \tag{10}$$

This converts a dead satellite into a controlled re-entry rather than an uncontrolled debris source—directly mitigating Kessler cascade risk.

Communication & Blackout Handling

Line-of-Sight Computation

A satellite can receive commands only when it subtends an elevation angle $\varepsilon \geq \varepsilon_{\min}$ above at least one of the six ground stations. The elevation is computed in ECEF after rotating the ECI position by GMST:

$$\varepsilon = \frac{\pi}{2} - \arccos\left(\frac{(\mathbf{r}_{\text{sat}} - \mathbf{r}_{\text{gs}}) \cdot \hat{z}_{\text{gs}}}{|\mathbf{r}_{\text{sat}} - \mathbf{r}_{\text{gs}}|}\right) \tag{11}$$

GMST is computed via the IAU Earth Rotation Angle formulation, referenced to the corrected J2000 epoch ($t_0 =$ 946,727,935.816 s Unix).

Store-and-Forward Uplink Queue

If LOS is unavailable at maneuver scheduling time, the burn sequence is placed in a `uplink_queue`. On every simulation step, queued burns are tested against the current satellite position; as soon as LOS is re-established the commands are transparently promoted to the live `maneuver_queue` and executed at their original scheduled times. This eliminates the "blind conjunction" failure mode described in the problem statement.

Maneuver Validation & Constraints

Every externally submitted maneuver sequence is validated against all physical and operational constraints *before* being committed:

Constraint	Value
Communication latency	$\geq +10$ s from now
Thruster cooldown	≥ 600 s between burns
Max $\lvert\Delta\mathbf{v}\rvert$ per burn	$\leq 15\ \mathrm{m\,s^{-1}}$
Fuel sufficiency	$m_{\text{post}} \geq m_{\text{dry}}$
Ground station LOS	at least one station visible

Validation is performed iteratively across the entire sequence, updating the projected mass after each burn, so a sequence of individually valid burns that *collectively* exhaust the fuel is correctly rejected.

Real-Time Frontend

The **Orbital Insight** dashboard is built with TypeScript and rendered via WebGL / Canvas for stable 60 FPS at full debris density.

- **Ground Track Map** — Mercator projection with live constellation markers, 90-minute historical trails, predicted trajectory overlays, and a dynamic terminator line.
- **Conjunction Bullseye** — Polar chart centred on the selected satellite; radial axis encodes TCA, colour encodes miss-distance risk (green/yellow/red).
- **Telemetry Heatmaps** — Per-satellite fuel gauges and a Δv cost vs. collisions-avoided efficiency graph.
- **Maneuver Gantt** — Timeline of past and future burns with cooldown blocks and blackout-overlap warnings.

High-frequency position data is streamed via a **binary WebSocket** (7 `Float64` fields per object, little-endian, 10 Hz). Each frame transfers the full object population in $10{,}050 \times 7 \times 8 \approx 563$ kB

Complexity Summary

Component	Complexity	Wall time
Telemetry ingest	$O(N)$	< 1 ms
RK4 propagation (parallel)	$O(N/P)$	~ 5 ms
Conjunction filter (P1–P5)	$O(N_s \cdot N_d)$	< 5 ms
Brent TCA (parallel)	$O(S \cdot I/P)$	~ 50 ms
Evasion + recovery scheduling	$O(T)$	< 1 ms
Full 24h screening	—	$<$ **100** ms

$N = 10{,}050$ objects, $N_s = 50$ satellites, $N_d = 10{,}000$ debris, $S =$ surviving pairs, $I =$ Brent iterations, $P =$ CPU cores.

Conclusion

Hyperion demonstrates that fully autonomous constellation management is achievable within tight latency budgets using principled algorithm selection rather than brute-force compute. The key insight is that a five-phase *funnel*—reducing 5×10^5 candidate pairs to a handful before invoking the expensive Brent solver—achieves a 320× **speedup** over the naïve grid without sacrificing accuracy. Combined with the store-and-forward uplink system, trial-ephemeris burn selection, and autonomous EOL management, Hyperion provides a robust, fuel-optimal, and zero-collision operational posture for large LEO constellations.

Repository: https://github.com/RohanJ2006/hyperion
Docker: `docker build . && docker run -p 8000:8000`
License: MIT